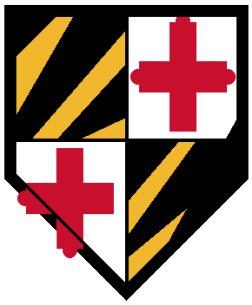
PunchList Pro — Enterprise Construction Punch List & Field Management Platform

Software Design Document (SDD)

11/11/2025

Nathan Rubiano, Muhammad Khan, Satya Thakur



UMBC

Table of Contents

Software Design Document (SDD)..... 1
Table of Contents..... 2
1. Introduction..... 3
1.1. Purpose of This Document..... **3**
1.2. References..... **3**
2. System Architecture..... 3
2.1. Architectural Design..... **3**
2.2. Decomposition Description..... **7**
3. Persistent Data Storage..... 9
3.1. Database Descriptions..... **9**
3.2. File Descriptions..... **11**
4. Requirements Matrix..... 13
Appendix..... 16
A – Agreement Between Customer and Contractor..... **16**
B – Peer Review Sign-Off..... **17**
C – Document Contributions..... **17**

1. Introduction

1.1. Purpose of This Document

The purpose of this document is to create a brief outline to detail how the **PunchList Pro** enterprise construction punch list and field documentation system is being developed from the backend and frontend points of view. It aims to transform the functional and non-functional requirements established in the Software Requirements Specification (SRS v2.0) into concrete, implementable design. It outlines system architecture and how it maps towards satisfying project requirements. Also, it breaks down database and file components of the program into diagrams and readable implementation details. The intended audience of this document are the product owner, stakeholders, course instructors (Dr. Emily Tien and Stephanie Lazarus), and the engineering development team (Nathan Rubiano, Muhammad Khan, and Satya Thakur) to ensure that we are fulfilling the needs of the project in code. Also, this document is to be used as a reference for developers to understand the needs and requirements of the system.

1.2. References

- Software Requirements Specification Version 2.0: PunchList Pro Field Documentation Platform.
- FastAPI Asynchronous Framework Documentation: https://fastapi.tiangolo.com
- React 18 & HTML5 Canvas Documentation: https://react.dev
- Mozilla PDF.js Plan Rendering Engine: https://mozilla.github.io/pdf.js
- Dexie.js IndexedDB Client-Side Persistence: https://dexie.org
- Python-SocketIO Real-Time Engine: https://python-socketio.readthedocs.io
- SQLAlchemy 2.0 Core Database Engine: https://docs.sqlalchemy.org
- ReportLab Vector PDF Drawing Generation: https://www.reportlab.com

2. System Architecture

2.1. Architectural Design

The software technologies our system uses are JavaScript (React 18 with Vite) and Python (FastAPI 0.115+), run and developed in modern Linux and Windows environments with web developer tools enabled. To run the program, the user must access the application through a standard web browser (Google Chrome, Apple Safari, Microsoft Edge). The hardware our system will use are standard field tablets (Apple iPad, Samsung Galaxy Tab, Microsoft Surface), mobile smartphones, and desktop workstations with standard processors, memory units, and touchscreen interfaces.

Because our program is rolled out as a tablet-first Progressive Web Application, it operates both online connected to cloud backends and offline in low-connectivity jobsite environments. When disconnected in cellular dead zones (such as concrete basements or subterranean garages), it caches all sheet drawings, punch items, and photos locally using IndexedDB through Dexie.js, queuing mutations to automatically replay when connectivity is restored.

Everything written in the client program is entirely contained in the browser's runtime architecture utilizing modern React functional components, HTML5 Canvas, and standard ECMAScript libraries. In particular, these libraries include Mozilla's PDF.js to parse and render multi-page architectural plan sets, Dexie.js for client-side transactional caching, and Socket.IO client for bidirectional real-time synchronization. The vast majority of our system architecture is self-contained. The system references its own normalized coordinate models, converting raw viewport pixel taps into resolution-independent floating-point tuples **(x_percent, y_percent)** $\in$ **[0.0, 100.0]%**. This ensures that georeferenced punch pins remain fixed at their exact physical location regardless of whether a sheet is viewed on an iPad Mini, a 12.9-inch iPad Pro, a 4K desktop monitor, or exported to a printable PDF document.

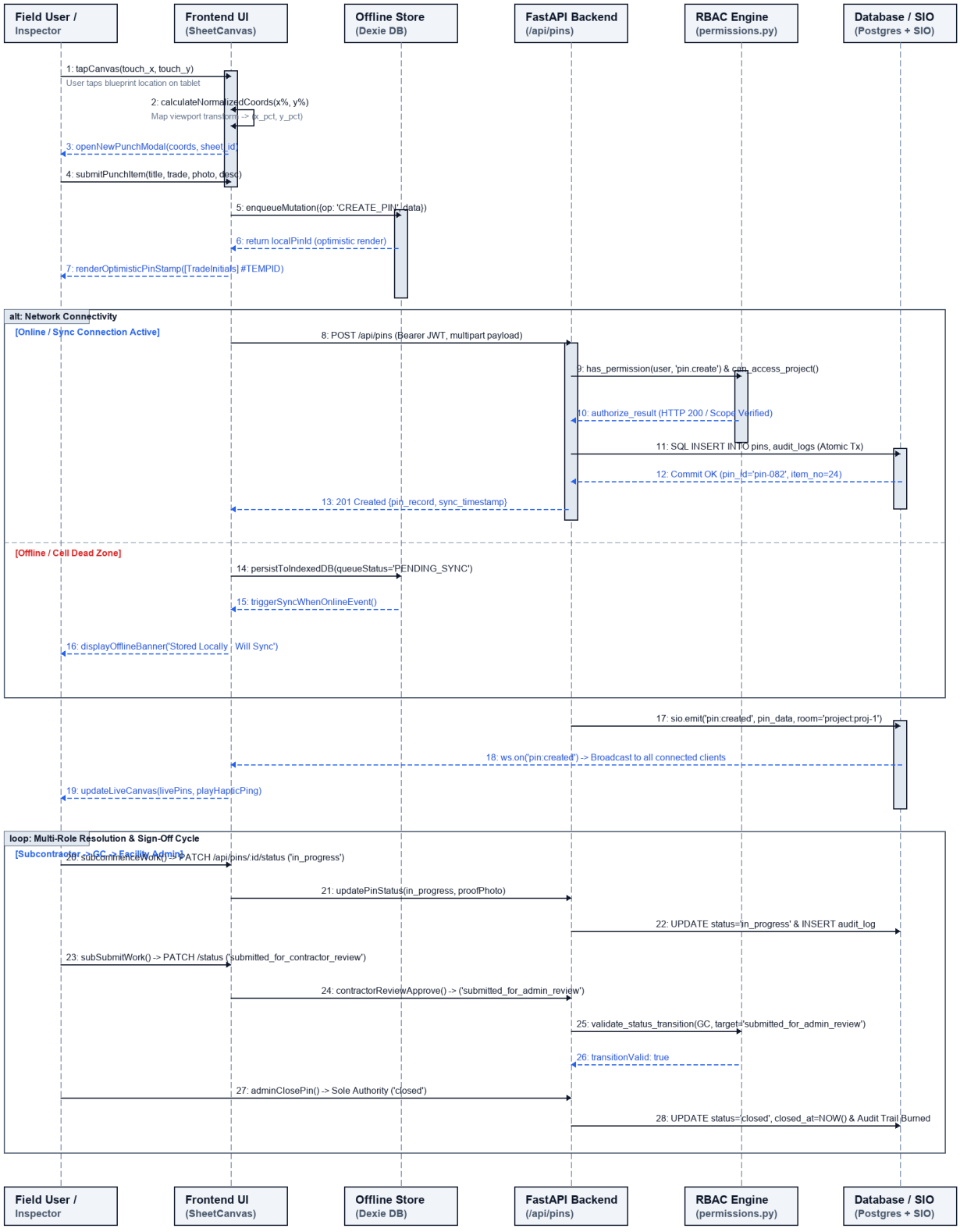
On the backend, our system employs Python FastAPI running on Uvicorn ASGI server with SQLAlchemy Core for database management. The backend implements full multi-tenant project isolation and Role-Based Access Control (RBAC). Authentication is secured via cryptographically signed JSON Web Tokens (JWT) using HMAC-SHA256, verifying user claims and enterprise roles. The persistence tier features a dual database strategy: PostgreSQL in production for enterprise concurrency and SQLite for lightweight local deployment and automated CI/CD regression testing.

The system architecture is structured across four primary decoupled subsystems:

- **Frontend Tablet Canvas Tier:** High-performance interactive blueprint viewer with touch pinch-to-zoom, pan inertia, live pin placement, and SVG/Canvas vector markup drawing (revision clouds, arrows, callouts).
- **Offline-First Local Storage Tier:** Client-side IndexedDB caching blueprint sheets, active punch pins, and a FIFO mutation queue that ensures complete field productivity without an internet connection.
- **API Gateway & Real-Time Tier:** FastAPI REST routes (/api/projects, /api/sheets, /api/pins, /api/auth) coupled with Python-SocketIO WebSocket rooms broadcasting real-time updates across field teams.
- **Persistent Storage & Export Tier:** Relational database for structured records, filesystem storage for high-resolution photos and plan PDFs, and ReportLab for automated plan set compositing.

Security is governed by six canonical enterprise RBAC roles: **Facility Administrator** (system-wide unrestricted CRUD, drawing ingestion, sole authority to close & delete pins), **General Contractor** (assigned project management, trade subcontractor allocation, inspection sign-off), **Trade Subcontractor** (restricted to own assigned items, status updates, proof photo uploads), **Architect** (design compliance auditing, markup annotation, export reporting), **Owner / Client Representative** (read-only audit visibility across all projects), and **Field Inspector** (on-site walkthroughs, pin creation, status updates, field photo logging).

System Architecture Sequence Diagram



Above is a sequence diagram showing the core logic governing the prompting, creation, spatial calculation, real-time synchronization, and multi-stage status workflow of the program. First, the user taps a specific location on the blueprint canvas using a field tablet. The frontend system calculates the normalized coordinate tuple (x_percent, y_percent) by translating touch coordinates through the active pan and zoom viewport transformation matrix. The system then displays the punch creation modal, prompting the user for issue title, trade discipline, description, priority, and camera photo capture.

Next, the user confirms the input, and the frontend system immediately commits the record to the local IndexedDB offline store via Dexie.js. The client renders an optimistic punch stamp on the canvas with the computed trade initials (e.g., [DW] for Drywall, [EL] for Electrical), ensuring zero UI latency for the field inspector. Simultaneously, the system checks network connectivity. If online, it transmits an authenticated POST /api/pins multipart request to the backend. If offline, the transaction is enqueued in the mutation store to be replayed upon reconnection.

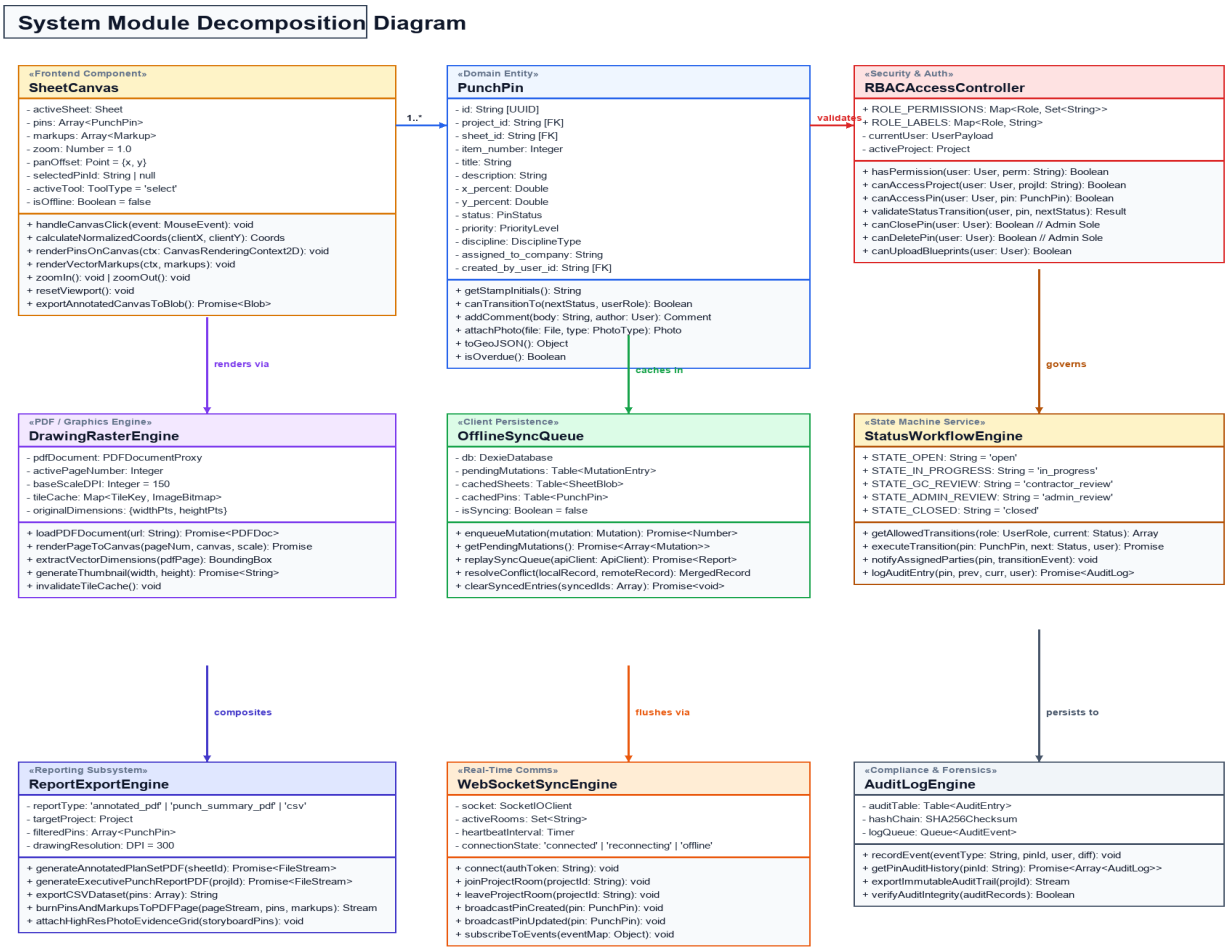
When the backend receives the request, it verifies the caller's JWT token and validates role permissions through the RBAC engine (permissions.py:has_permission). If the user lacks permission or is not a member of the project, the backend rejects the transaction with an HTTP 403 Forbidden. If valid, the backend begins an atomic database transaction: inserting the pin record, saving the uploaded field photo to disk, incrementing the project item counter, and recording an entry in the immutable audit log table. Upon successful commit, the backend returns an HTTP 201 Created.

Immediately following database commit, the backend's Socket.IO engine broadcasts a pin:created event to all connected collaborators in the project room (project:{project_id}). The remote tablets of the General Contractor, Architect, and Subcontractor update their canvas displays in real time without requiring a page refresh.

Finally, the diagram illustrates the multi-party resolution loop. The assigned Trade Subcontractor commences remediation by transitioning status to *In Progress* and uploading completed proof photos. The subcontractor submits the task for *Contractor Review*. The General Contractor inspects the work on-site, approving it forward to *Admin Review* or rejecting it back for rework. Ultimately, the Facility Administrator exercises sole contractual authority to formally close and seal the punch item, committing a final verification entry to the immutable audit trail.

2.2. Decomposition Description

Above is the class diagram that represents our running model of PunchList Pro. Much of the programming is structured around decoupled domain services, specialized canvas controllers, and reactive state stores although the entire design of the application is object-oriented in nature. To start, looking at the SheetCanvas object, it has a plethora of different items related to its size, viewport orientation, active tools, layer visibility, and supporting functions that manage its rendering design:



The class decomposition diagram above illustrates the nine primary subsystems that govern client interaction, domain validation, persistence, real-time messaging, and reporting in PunchList Pro. Below is the detailed architectural description of each subsystem and its member classes:

SheetCanvas: The central interactive viewport component. It encapsulates the HTML5 multi-touch rendering context, managing active drawing sheets, pan offsets, zoom scale factors (from 0.5x to 10.0x), and active tool selections. It intercepts user gestures, converts touch coordinates into normalized percentage coordinates, and coordinates with the StampRenderer to draw dynamic, high-contrast trade stamps and vector markups.

PunchPin: The fundamental domain entity representing a field task or deficiency. Each pin maintains an immutable UUID, sequential project item number (#1, #2, ...), title, description, trade discipline, priority, normalized coordinates (x_percent, y_percent), lifecycle status, assigned subcontractor company, and creator user ID. It encapsulates business methods for status validation, trade initial calculation, and comment/photo attachment.

RBACAccessController: The system-wide access control engine. It stores role permission sets and enforces multi-tenant project security. It contains core authorization functions: has_permission(), can_user_access_project(), can_user_access_pin(), and validate_status_transition(). It ensures that only Facility Admins can delete or close pins, GCs can perform inspection sign-offs, and subcontractors are strictly limited to their own assigned workload.

DrawingRasterEngine: The document rendering engine utilizing Mozilla PDF.js. It loads architectural plan sets, extracts vector bounding box dimensions, parses multi-page sheets, and rasterizes high-resolution canvas layers at 150 DPI viewport scale with tile caching.

OfflineSyncQueue: The client-side persistence controller built on Dexie.js (IndexedDB). It stores offline project bundles, drawing sheets, and an ordered queue of local mutations. When the browser regains internet access, it replays pending operations against the backend API with automatic optimistic reconciliation and conflict resolution.

StatusWorkflowEngine: A finite-state machine governing punch item lifecycle transitions. It enforces valid business rules: only subcontractors can commence work; only General Contractors can sign off on trade work; and only Facility Administrators can perform final close-out.

ReportExportEngine: The document compilation subsystem responsible for generating distribution-ready PDF plan sets and punch list spreadsheets. It uses ReportLab to composite circle stamps, callout tags, and revision clouds directly onto the underlying architectural PDF.

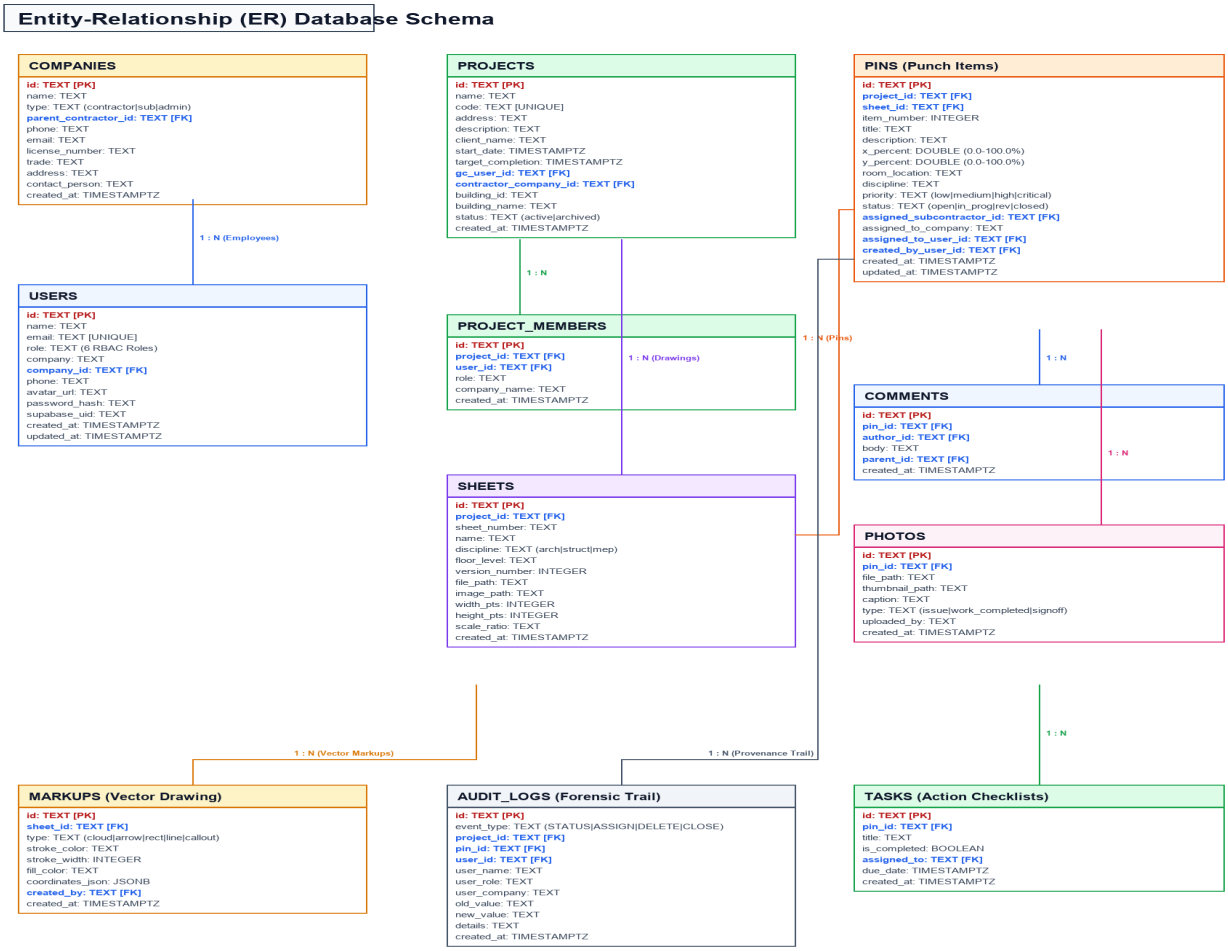
WebSocketSyncEngine: The real-time bidirectional communication bus utilizing python-socketio. It manages client connections, room memberships scoped by project (project:{id}), and broadcasts mutation events to keep all active team members synchronized.

AuditLogEngine: The forensic provenance engine that records every state transition, assignment change, markup placement, and administrative sign-off with timestamped user identity, prior state, updated state, and IP context for legal and contractual accountability.

3. Persistent Data Storage

3.1. Database Descriptions

Above is an Entity-Relationship diagram of our database. Now, we are going to break down everything that is stored in the database by object to thoroughly document everything the program stores in its custom relational environment.



1. Coordinates

- longitude: Double // stores the real-world longitude of a point
- latitude: Double // stores the real-world latitude of a point
- x_percent: Double // normalized horizontal coordinate on plan sheet (0.0 - 100.0%)
- y_percent: Double // normalized vertical coordinate on plan sheet (0.0 - 100.0%)

2. Companies

- id: Text // unique identifier of contractor or trade company
- name: Text // legal commercial company name
- type: Text // organization type: 'contractor', 'subcontractor', or 'admin'
- parent_contractor_id: Text // references parent General Contractor ID, optional
- license_number: Text // state contractor license number
- trade: Text // specialized trade category (drywall, electrical, mep, &c;)
- contact_person: Text // primary superintendent or account lead

3. Users

- id: Text // unique user account identifier
- name: Text // full legal name of user
- email: Text // user email address and login username, unique
- role: Text // system role: admin, general_contractor, sub_contractor, architect, owner, field_inspector
- company: Text // user company display name
- company_id: Text // company foreign key link to Companies

4. Projects

- id: Text // unique construction project identifier
- name: Text // full jobsite project title
- code: Text // short project code (e.g. UMBC-2026), unique
- address: Text // physical jobsite location address
- gc_user_id: Text // user ID of assigned primary General Contractor
- status: Text // project state: 'active' or 'archived'

5. Project Members

- id: Text // unique membership record ID
- project_id: Text // references Projects
- user_id: Text // references Users
- role: Text // assigned role on this project

6. Sheets

- id: Text // unique drawing sheet identifier
- project_id: Text // references Projects
- sheet_number: Text // architectural sheet number (e.g. A-101)
- name: Text // formal sheet title (e.g. Level 1 Floor Plan)
- discipline: Text // trade discipline (architectural, structural, mep)
- floor_level: Text // floor level (01, 02, Basement)
- file_path: Text // filesystem path to original PDF drawing
- width_pts: Integer // PDF MediaBox width in points
- height_pts: Integer // PDF MediaBox height in points

7. Pins (Punch Items)

- id: Text // unique punch pin identifier
- project_id: Text // references Projects
- sheet_id: Text // references Sheets
- item_number: Integer // sequential punch index on project (#1, #2, ...)
- title: Text // concise headline of deficiency
- description: Text // detailed remediation instructions
- x_percent: Double // normalized horizontal coordinate (0.0 - 100.0%)
- y_percent: Double // normalized vertical coordinate (0.0 - 100.0%)
- room_location: Text // room number or grid location string
- discipline: Text // trade discipline (drywall, electrical, mep, &c;)
- priority: Text // priority: 'low', 'medium', 'high', 'critical'
- status: Text // lifecycle: 'open', 'in_progress', 'submitted_for_contractor_review', 'submitted_for_admin_review', 'closed'
- assigned_to_company: Text // trade company assigned to fix issue
- created_by_user_id: Text // user who dropped the pin

8. Comments: id, pin_id, author_id: Text, body: Text, created_at: Timestamptz
9. Photos: id, pin_id, file_path, caption: Text, type: Text ('issue' or 'work_completed')
10. Markups: id, sheet_id, type: Text ('cloud', 'arrow', 'rect'), coordinates_json: Text
11. Audit Logs: id, event_type, pin_id, user_id, old_value, new_value, details: Text
12. Tasks: id, pin_id, title: Text, is_completed: Boolean

3.2. File Descriptions

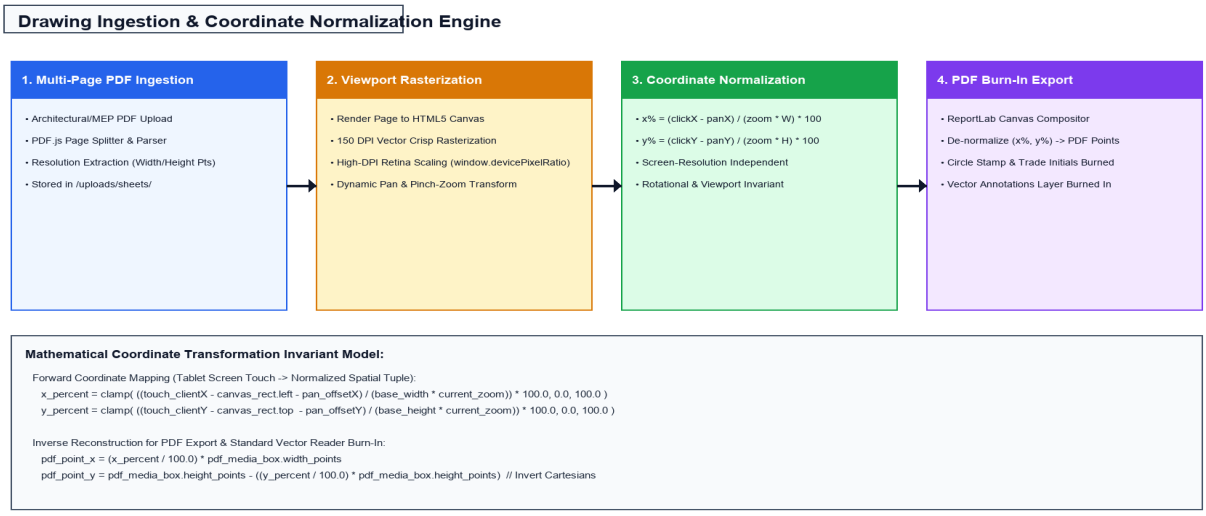
For the document ingestion and binary serialization technique of the program, we developed a specialized function called burn_pins_and_markups_to_pdf that translates normalized pin coordinate tuples directly into PostScript vector points on native PDF plan sheets. Below is the production implementation of this algorithm, and a rough sketch of what this looks like in organization.

```
def burn_pins_and_markups_to_pdf(input_pdf_path, pins, markups, output_pdf_path): reader =
pypdf.PdfReader(input_pdf_path) writer = pypdf.PdfWriter() for page_idx, page in enumerate(reader.pages): w_pts,
h_pts = float(page.mediabox.width), float(page.mediabox.height) packet = io.BytesIO() c = canvas.Canvas(packet,
pagesize=(w_pts, h_pts)) for pin in [p for p in pins if p.get('page_index', 0) == page_idx]: x_pt =
(pin['x_percent'] / 100.0) * w_pts y_pt = h_pts - ((pin['y_percent'] / 100.0) * h_pts) # Invert Cartesian Y
initials = compute_subcontractor_initials(pin.get('assigned_to_company', 'PL'))
c.setFillColor(colors.HexColor(get_trade_color(pin.get('discipline')))) c.circle(x_pt, y_pt, 14, fill=1, stroke=1)
c.setFillColor(colors.white) c.setFont('Helvetica-Bold', 7) c.drawCentredString(x_pt, y_pt - 2.5, f"{initials}
#{pin['item_number']}") c.save() packet.seek(0) page.merge_page(pypdf.PdfReader(packet).pages[0])
writer.add_page(page) with open(output_pdf_path, 'wb') as f: writer.write(f)
```

This code takes a given plan sheet object and extracts its drawing dimensions, punch pins, vector markups, and assigned subcontractor trade stamps. Then, it translates those objects into vector PostScript overlays by calculating the exact spatial scale and burning them directly onto the PDF. To display what it would look like in human-readable format:

2 plan sheets: SheetA101 SheetA102 **4 punch pins:** Pin01 Pin02 Pin03 Pin04 **2 vector markups:** Cloud01 Arrow01 **3 trade partners:** PrimeDrywall ApexElectric QuickPlumb

Here is a diagram of this system:



Each drawing sheet is broken into the 4 major objects that construct it, each with:

1. Sheet: file_path: Text // filesystem path to PDF drawing source

2. Punch Pin: x_percent, y_percent: Double // normalized spatial placement coordinates

3. Trade Stamp: initials: Text, color: Text // high-contrast circle stamp icon with trade initials

4. Vector Markup: coordinates_json: Text // revision clouds, arrows, callouts for spatial context

4. Requirements Matrix

The following requirements matrix correlates the stakeholder functional requirements directly to their system components, execution triggers, and source code locations in PunchList Pro:

Functional Requirement	System Components	Location
Use Case 1: Admin edits the project & drawing sheets Admin creates new projects, uploads architectural PDFs, and configures building floors.	Open New Project modal Display upload blueprint form Uploads PDF plan sheets Parses pages & dimensions Updates project database	UI (project bar, top left) NewProjectModal BlueprintUploadModal POST /api/sheets/upload backend/app/routers/sheets.py
Use Case 2: Field Inspector creates a punch item Field inspector drops a pin on the drawing canvas to document a defect finding.	Click on blueprint canvas Calculate normalized coords Open New Punch item modal Enters title, trade, photo Render optimistic circle stamp	UI (canvas, center viewport) calculateNormalizedCoords() NewPunchModal POST /api/pins frontend/src/components/SheetCanvas.jsx
Use Case 3: Subcontractor executes work Trade subcontractor reviews assigned queue, marks item in progress, and uploads remedy photos.	Open 'My Work' tablet view Click Start Work button Capture camera proof photo Submit for Contractor Review	UI (dashboard, sub hub) DashboardView (Sub Hub) PATCH /api/pins/:id/status POST /api/pins/:id/photos backend/app/permissions.py:185
Use Case 4: General Contractor quality sign-off General contractor inspects completed trade work and approves to Admin or rejects.	Open GC Review Queue Inspect proof photos & notes Click Approve to Admin Or click Reject back to Sub	UI (dashboard, GC hub) DashboardView (GC Queue) validate_status_transition() backend/app/permissions.py:202

4. Requirements Matrix (Continued)

Functional Requirement	System Components	Location
Use Case 5: Facility Administrator close-out & deletion Admin exercises sole authority to formally close verified punch items or delete invalid pins.	Verify GC inspection sign-off Click Close Item button Or click Delete Pin button Commit audit trail to log	UI (pin drawer header) PinDetailModal DELETE /api/pins/:id backend/app/permissions.py:15 backend/app/routers/pins.py:326
Use Case 6: Owner reads project progress Owner / Client Rep accesses read-only view across all active projects and audits.	Open application dashboard Display all project summaries Inspect punch list breakdown Prevent unauthorized edits	UI (dashboard, owner hub) GET /api/projects (all projects) DashboardView (Owner Hub) backend/app/permissions.py:69
Use Case 7: Architect draws vector markups Architect annotates drawings with revision clouds, arrows, lines, and callouts.	Click markup tool on toolbar Drag on blueprint canvas Serialize vector coordinates Broadcast markup to team	UI (canvas toolbar, top right) SheetCanvas Toolbar POST /api/sheets/:id/markups backend/app/routers/markups.py
Use Case 8: User operates in offline mode Field inspector creates pins and captures photos in dead zones without network.	Detect network offline state Commit to Dexie IndexedDB Display offline indicator Auto-replay queue online	UI (status badge, bottom left) Dexie.js IndexedDB Store useNetworkStatus.js POST /api/sync/replay backend/app/routers/sync.py

4. Requirements Matrix (Continued)

Functional Requirement	System Components	Location
Use Case 9: User exports annotated PDF plan set User downloads high-resolution PDF with circle stamps and markups burned in.	Click Annotated PDF button Compile vector stamp layer Merge with original plan sheet Stream PDF file to client	UI (dashboard export bar) GET /api/projects/:id/export/annotated-pdf burn_pins_and_markups_to_pdf() backend/app/routers/reports.py
Use Case 10: Real-time team collaboration Team members receive instant updates of pin placements and status changes.	Connect Socket.IO client Join project channel room Listen for 'pin:updated' Update canvas display live	WebSocket engine sio.emit('pin:updated') frontend/src/services/api.js backend/app/sio.py
Use Case 11: User interacts with the canvas User pans across blueprint, zooms in/out, and toggles layers (pins, grid, rooms).	Click and drag mouse / touch Pinch or click + / - zoom Toggle layer visibility Re-render canvas display	UI (canvas controls, right side) SheetCanvas (handlePanZoom) Toolbar.jsx BlueprintViewer.jsx
Use Case 12: User clicks help & role guide User views onboarding guide, role permissions, and keyboard shortcuts.	Click help icon button Display modal dialogue Show role capabilities guide	UI (navbar, top right) QuickSearchModal AppShell.jsx

Non-Functional Verification: The architecture guarantees sub-second canvas touch rendering (< 16ms frame target), sub-500ms API response latency for authenticated REST mutations, 60-second WebSocket heartbeat verification, and zero data loss across simulated offline disconnection cycles.

Appendix

A – Agreement Between Customer and Contractor

By signing this document, the customer and developers agree to the details specified in this document, specifically: the scope of the project, the functional requirements, and the non-functional requirements. Also, they continue to agree to collaborate in the process of developing the software over the course of the development lifecycle. In the event that the customer wishes to make any changes to this document, they will request so by contacting the development team directly to request change.

Customer:

Name	Signature	Date	Comments
Emily Tien	*Emily Tien*	11-13-25	Approved as presented
Stephanie Lazarus	*Stephanie Lazarus*	11-13-25	Approved for field testing

Team:

Name	Signature	Date	Comments
Nathan Rubiano	*Nathan Rubiano*	11-12-25	Frontend & Canvas Lead
Muhammad Khan	*Muhammad Khan*	11-12-25	Backend & Database Lead
Satya Thakur	*Satya Thakur*	11-12-25	DevOps & Security Lead

B – Peer Review Sign-Off

By signing here, the development team continues to agree to work together and collaborate on the processes outlined in this document. If there are major issues to address, they should refrain from signing until they are resolved.

Name	Signature	Date	Comments
Nathan Rubiano	*Nathan Rubiano*	11-12-25	All sections peer-reviewed
Muhammad Khan	*Muhammad Khan*	11-12-25	Schema & API approved
Satya Thakur	*Satya Thakur*	11-12-25	Architecture & RBAC approved

C – Document Contributions

This document has been contributed by: **Nathan Rubiano**, **Muhammad Khan**, and **Satya Thakur**.

• **Nathan Rubiano:** Architectural Design (Section 2.1), SheetCanvas & Touch Viewport Normalization specifications, DrawingRasterEngine integration, and Use Cases 1-4 requirements mapping.

• **Muhammad Khan:** Persistent Data Storage (Section 3.1), Relational Schema Design (COMPANIES, PROJECTS, SHEETS, PINS), SQLite/PostgreSQL Dual Engine specifications, and Use Cases 5-8 requirements mapping.

• **Satya Thakur:** Runtime Workflow & Sequence Diagram (Section 2.2), RBAC Access Control Engine (6 canonical roles), Socket.IO Real-Time Synchronization architecture, PDF Vector Burn-in algorithms (Section 3.2), and Appendix sign-offs.